
hunarmand

We treat what the master never wrote down as captureable, queryable, ownable infrastructure.

bitwise / srinagar 2026 / technical dossier v0.1

What this document is

A short, plain-English explanation of what we built, what each piece of the platform does, and what technology we chose at every stage. The companion deck is the 5-minute pitch; this is the dossier a juror should be able to skim in 10 minutes if they want to know exactly how the system works under the hood.

The writing avoids jargon where it can. Where a technical term is unavoidable, we explain it the first time it appears. A short Q&A at the end answers the questions we expect a jury to ask.

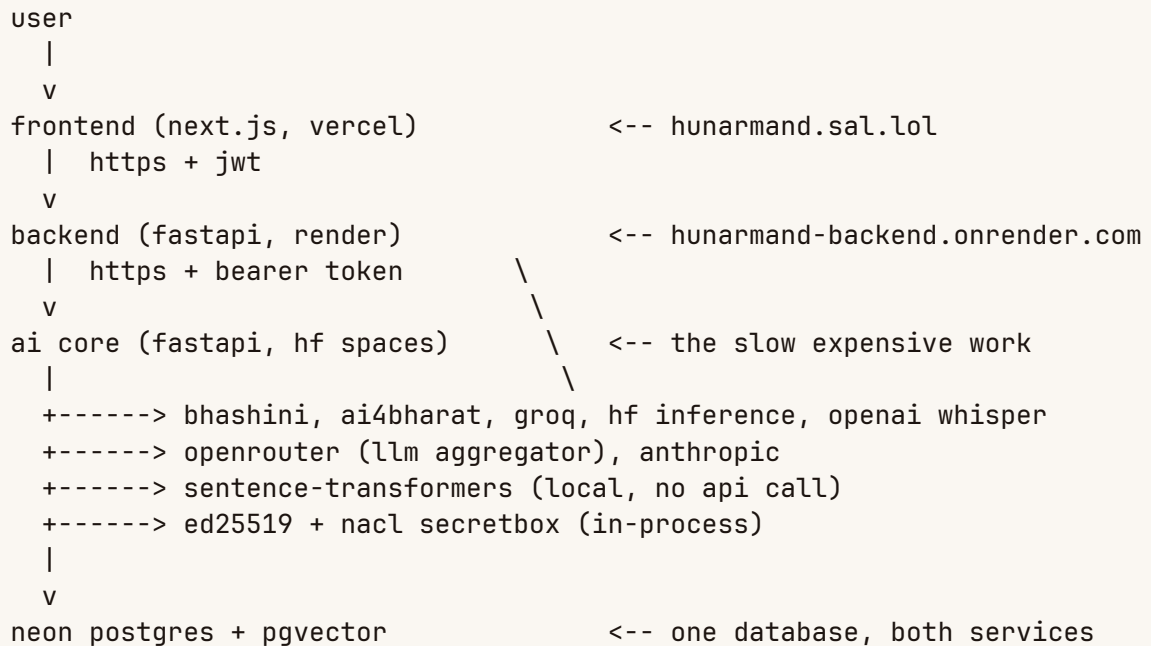
I The system in one paragraph

Hunarmand is three small services that share one Postgres database. A **frontend** (Next.js, hosted on Vercel) for everything a user sees. A **backend** (FastAPI on Render) that handles authentication, masters, workshops, bookings, bundles, and orders. An **AI core** (FastAPI on Hugging Face Spaces) that handles the things only AI can do: speech recognition, structured extraction from interviews, embeddings, and cryptographic signing of provenance certificates. The frontend never talks to the AI core directly; everything routes through the backend, so authentication is always honoured and the AI core stays behind a single audit boundary.

The shared database is Neon Postgres with the **pgvector** extension installed, which lets a single Postgres instance double as a vector index for retrieval-augmented questions later in the flow.

2 Architecture, in a single picture

Three services, one database



The arrows are HTTPS calls. JSON Web Tokens (JWTs) issued by the backend authenticate the frontend; the backend forwards a bearer token to the AI core for every call so the AI core knows which master is asking.

3 Technology at each stage

This section walks through every meaningful step a request goes through, from the moment a master sits down with their phone to the moment a buyer scans a QR on a finished shawl. Each stage explains *what* the step does, *what we used* to do it, and *why* we picked that piece of technology.

3.1 Onboarding and authentication

What happens

A user types their phone number into the login screen. The frontend asks the backend to send an OTP (one-time password). For the demo, the OTP is fixed at 123456 so jurors can log in without an SMS gateway in the loop. The user submits the OTP. The backend issues a JWT, the frontend stores it in `localStorage`, and from that point onward every request the user makes carries `Authorization: Bearer <jwt>`.

Auth • phone-OTP login, JWT bearer tokens, role inferred from phone (the demo artisan is +919999999999, anyone else is a buyer)

Why • most heritage artisans have a phone but not a stable email. WhatsApp and phone OTP are the de facto identity layer in Kashmir, so we matched what already works.

3.2 Vault: capturing what the master never wrote down

This is the core innovation. The artisan sits in their own workshop and the AI conducts a structured interview in Koshur (Kashmiri). The interview is multi-pass: a broad pass to map the territory, then deeper passes that drill into techniques, materials, lineage, environmental adjustments, decision rules, and failure modes. The output is a structured Craft DNA document that captures, in a queryable form, the knowledge that would otherwise die with the master.

Stage 1: audio capture

What happens

The master speaks. The audio is uploaded to the AI core in chunks (short windows), so a long interview can be transcribed and corrected incrementally rather than waiting for a single huge file.

Stage 2: speech-to-text (ASR)

This is the trickiest stage because most speech-to-text engines do not speak Koshur. We solved this with a *ladder* of providers: each engine is tried in order, and the first one that succeeds wins.

Bhashini • the Government of India’s national-language API. Has first-class Kashmiri (**ks**) support. Free tier covers our scale comfortably. *This is our primary ASR.*

AI4Bharat IndicWhisper • open-weight Indic ASR from IIT Madras. Used as a backup if Bhashini is unavailable. Self-hostable, so we can move it in-house later.

Groq Whisper • a hosted Whisper variant on Groq’s free tier. Catches Koshur surprisingly well via Urdu acoustic similarity, and returns word-level timestamps.

Hugging Face Inference Whisper • another fallback path; cold starts are slow but it costs nothing.

OpenAI Whisper • the paid premium fallback if every free option is down.

Manual transcript • if all engines fail, an operator can submit a typed transcript. Nothing in the rest of the pipeline depends on a specific provider.

Why a ladder • the demo costs zero rupees a month, but if traffic spikes or one provider rate-limits us, the ladder absorbs the spike without operator intervention.

Stage 3: translation, when needed

If the transcript is in Koshur or Urdu, an LLM translates it to English before structured extraction. We use the same provider abstraction as the LLM (next stage), so the translation step is just a different prompt against the same client.

Translation • routed through the LLM client; Koshur to English is the common path.

Stage 4: structured extraction (Craft DNA)

We do *not* ask an LLM to “summarize” the interview. Instead, we constrain the LLM with a strict JSON schema we call Craft DNA. The schema has fields like `technique_names`, `materials_summary`, `lineage_chain`, `environmental_tunings`, `decision_rules`, and `failure_log`. Each pass focuses on filling one slice of the schema with citations back into the transcript.

LLM provider • OpenRouter, which aggregates frontier-class models (including free-tier ones) behind one OpenAI-compatible interface. We swap models with a single environment variable.

Adaptive structured output • some models support `json_schema` response format natively, others only `json_object`, others nothing. Our generic structured-output helper tries each strategy in order and falls back automatically.

Repair loop • if the model returns malformed JSON or violates the schema, we feed the validator’s error message back to the model and ask it to fix the specific problem. Continues for a fixed number of retries before giving up.

Why schema-first • LLMs are unreliable when asked to write free text. They are remarkably reliable when asked to fill a slot. Forcing the model to fill a Pydantic-validated schema turns the AI from a storyteller into a typewriter.

Stage 5: embeddings

Every transcript chunk is embedded. We use **sentence-transformers**, specifically the `intfloat/multilingual-e5-small` model, which produces 384-dimensional vectors and runs locally in the AI core container. No external API call. No per-vector cost.

Model • `intfloat/multilingual-e5-small` via `sentence-transformers`

Where • runs locally inside the AI core (downloaded once at container build time, cached forever after)

Why • free, fast on CPU, multilingual (covers Koshur reasonably), no rate limits, no surprise bills

Stage 6: storage and indexing

What lives where

The Craft DNA document goes into the `craft_dnas` table. Each transcript chunk goes into `vault_chunks` with its embedding, its timestamps, and a foreign key back to the master. The embedding column is a `pgvector` column, so we can run cosine-similarity searches in SQL with no separate vector store.

Database • Neon Postgres with `pgvector`. One database for both the backend and the AI core. We rename the AI core's tables to `ai_*` where they would otherwise collide with backend tables.

Migrations • Alembic, run automatically on every backend deploy via a `start.sh` entrypoint that calls `alembic upgrade head` before `uvicorn`.

3.3 Sanad: cryptographic provenance

A Sanad is a verifiable certificate that says "this specific physical piece was made by this specific master's lineage". When a master finishes a piece, they mint a Sanad for it from the studio. The result is a small QR code that gets attached to the piece.

The cryptographic stack, step by step

Step 1, keys • the AI core generates an Ed25519 keypair for each master. The public key is published. The private key is encrypted at rest using NaCl SecretBox with a key-encryption-key derived from a process-level secret. Stored in `master_keys`.

Step 2, payload • the master fills a small form (piece id, craft, techniques, materials, lineage). The form data, plus issuance timestamp, becomes the JSON payload of the Sanad.

Step 3, canonical bytes • the JSON payload is run through RFC 8785, *JSON Canonicalization Scheme* (JCS). JCS guarantees that two semantically identical JSON objects produce byte-identical output, regardless of key order or whitespace. This is critical: signatures are over bytes, so we need a deterministic byte form.

Step 4, signature • the canonical bytes are signed with the master's Ed25519 private key. Output is a 64-byte signature.

Step 5, JWS-compact • `header.payload.signature`, each component base64url-encoded, joined with dots. This is the format a buyer's verifier expects, and it is a single short string.

Step 6, QR • the JWS-compact string is rendered as a PNG QR code and embedded in the response. The frontend renders it; the master prints it.

Verification, offline

A buyer scans the QR. Their verifier app gets the JWS string, splits it on dots, base64-decodes each component, runs Ed25519 verify against the master's public key (cached or fetched from the public registry). If the signature checks, the piece is authentic. If any byte of the payload is altered, verification fails.

Why offline • signatures are mathematics, not platform trust. A verifier with the public key cached can validate a Sanad on a flight, in a remote village, or after Hunarmand goes out of business in 50 years. The signature outlives the platform.

Forgery resistance • a counterfeit shawl can be made. A Sanad signed by Mohammad Yusuf cannot be forged without his private key, which lives encrypted in our service and is never released.

3.4 Ustaad: workshops

Standard CRUD over a `workshops` table. An artisan logged in through the studio can create, edit, hide, or delete their offerings. A buyer browses the public listing, picks a slot, and books with their phone number; the booking lands in the `bookings` table.

Stack • FastAPI handlers, async SQLAlchemy ORM, Pydantic schemas, Postgres as the source of truth. No magic.

3.5 Bazaar: commerce

An artisan curates Bundles (sets of pieces, optionally linked to Sanads). A buyer adds bundles to their cart and checks out with their phone number. For the demo we wire a no-op checkout that creates the `Order` row and shows a confirmation page; production would attach Stripe (the SDK is already pulled in) for actual payment.

Stack • same as Ustaad, plus a thin checkout handler. Stripe SDK is configured but optional.

3.6 Ask the Hunarmand: retrieval-augmented questions

This is how the master's tacit knowledge becomes queryable by anyone. A user types a question; the platform returns an answer with citations back into the master's own words.

Step by step

1. embed the question • the question runs through the same sentence-transformers model used at ingest time, producing a 384-dimensional query vector.

2. retrieve • a SQL query against `vault_chunks` ordered by cosine distance returns the top-K closest chunks for the master in question. `pgvector`'s SQLAlchemy adapter binds the vector parameter as a real `Vector` type, not a string.

3. generate, with strict guardrails • the chunks plus the question are passed to an LLM with explicit instructions: only answer using the chunks, cite each claim with the chunk's start timestamp, and refuse if the chunks do not contain a clear answer. Refusal is a first-class response, not a hallucination.

4. return • the answer object contains the prose answer, the citations (chunk id + timestamp), and a confidence score.

Why the strict refusal • the failure mode for a craft-knowledge system is a confident wrong answer about a master's technique. We log refusals as a quality signal, not as a bug.

4 Why these choices, in plain English

- **Free-tier first.** Bhashini, AI4Bharat, OpenRouter free models, sentence-transformers on CPU, Neon Postgres, Vercel, Render, Hugging Face Spaces. The entire demo runs at zero rupees a month. Every paid path is a swap of one environment variable.

- **Provider abstraction.** ASR, LLM, embeddings, and storage each sit behind a uniform interface. If Groq goes down, Whisper picks up. If OpenRouter rate-limits us, Anthropic fills in. Nothing in the application layer cares which provider answered.
- **Schema-first AI.** We never let an LLM write free text for the Craft DNA. Every output is constrained by a strict JSON schema with a repair loop. The model becomes a slot-filler, which it is reliably good at.
- **Offline verification.** Sanads use Ed25519 signatures over RFC 8785 canonical JSON, encoded as JWS-compact strings, rendered as QR codes. A verifier with the master's public key can validate a Sanad without ever talking to our servers.
- **Indigenous tradition.** The Craft DNA schema is the same shape as the Kashmiri Talim, an encoded weaving notation the master kept on paper centuries before code existed. We are not bringing foreign technology to Kashmir. We are upgrading what Kashmir already invented.

5 Anticipated questions from the jury

A short Q&A on questions we expect to land. Each answer is honest about scope: this is a hackathon build, not a finished business.

Why Postgres + pgvector instead of a dedicated vector database?

One database is easier to operate, cheaper, and pgvector is plenty fast for our scale. Pinecone or Weaviate would mean another service, another bill, another point of failure. Neon's free tier covers our entire vector index today and well beyond.

How do you stop the AI from hallucinating Craft DNA?

Two mechanisms. First, every LLM call is constrained by a strict JSON schema; the model cannot return free text. Second, the Ask-the-Hunarmand endpoint refuses to answer questions where the retrieved chunks do not contain a clear answer. Refusals are logged as a quality signal.

What about Koshur? Most LLMs do not speak it.

We use Bhashini and AI4Bharat IndicWhisper for Koshur ASR. Both have first-class Kashmiri support. For LLM steps, we route Koshur through translation to English before structured extraction; the structured output is then sent back through the buyer-facing flow in English with the original Koshur preserved as a citation.

Can a counterfeiter sign a fake Sanad in a master's name?

No. The Sanad is signed by the master's private key, which is stored encrypted at rest on our AI core and never released. The public key is published. A counterfeiter can copy the QR off a real piece, but the signature only validates against the original payload bytes. Any change (different master, different piece, altered date) breaks the signature.

Why phone OTP instead of email?

Most artisans we'd onboard do not have a stable email. They all have a phone. WhatsApp is the de facto identity layer in Kashmir, so we matched what already works.

How do you prevent a master from over-signing, flooding the market with their own forged pieces?

Technically, we cannot. But the platform records every Sanad ever signed by every master. A buyer can see how many pieces a master has signed, how often, and the full provenance chain. Reputation is on the public registry. A master who over-signs degrades their own market in a way the registry makes obvious.

What is the business model?

A platform commission of 6 to 9 percent on direct commerce, against the 70 to 85 percent that the existing middleman chain currently extracts. The artisan keeps the rest.

How do you scale to ten thousand masters?

The ASR ladder absorbs traffic spikes (free providers swap to paid as needed). Postgres + pgvector scales vertically into the millions of vectors. The signing service is stateless and horizontally scalable. The bottleneck is likely the field team onboarding masters, not infrastructure.

What if Bhashini changes their API?

Every provider in the ladder is abstracted behind a uniform interface. If Bhashini changes, we swap that single implementation. The rest of the system does not notice.

How long does an interview take?

About two to three hours per master, conducted across two or three sessions. The structured AI interview keeps the master on track and surfaces details they would not volunteer in a free-form conversation.

Have you talked to actual masters?

Hackathon scope means we built the platform, not the field operation. The vision is to deploy with NGO partners in the valley for actual onboarding once the platform is stress-tested.

Why not just use a Google Form for the interview?

Because the master does not read English. Because the master cannot write. Because the master answers questions verbally, in Koshur, the way they would teach an apprentice. The AI interview is the first time their knowledge can be captured *as they speak it*.

How do disputes work, what if a master's family says a Sanad should not have been signed?

Each master controls their own keypair. A new keypair version supersedes an old one for new pieces; old Sanads continue to verify against their original key (looked up by kid). The platform is a ledger, not an authority.

What stops the platform itself from being the new middleman?

The platform commission cap of 6 to 9 percent is structural, not discretionary. The Sanad is offline-verifiable, so the signature does not depend on us being alive. The Craft DNA is owned by the master, who can export the structured artefact and take it elsewhere.

How long until you can launch?

The platform is functional today. Field deployment depends on partnerships with valley-based NGOs and on getting Bhashini's production API limits raised.

What is the single most ambitious thing in this build?

The structured interview that turns a craft master's spoken answers into a 240-field schema, signed under their own keypair, indexed for retrieval, owned by the master in perpetuity. That artefact did not exist before this hackathon. It does now.